Homework 08

Rahaf Alabdullah (rahafaa2)

Question 1: Boundary and margin

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path
data_dir = Path(__file__).parent / "data"
fig_dir  = Path(__file__).parent / "figures"
df = pd.read_csv(data_dir / "margin-geometry-20.csv")
x1, x2, y = df["x1"].to_numpy(), df["x2"].to_numpy(), df["y"].to_numpy()
beta = np.array([0.8, 0.6])
f = beta[0]*x1 + beta[1]*x2
distance = np.abs(f) / np.linalg.norm(beta)
margin = y * f

cond1= np.isclose(margin, 1) #on
cond2= margin < 1 #inside
position = np.select([cond1, cond2], ["on", "inside"], default="beyond")
table = pd.DataFrame({"observation": df["observation"], "y": y, "f(x)": f, "distance": distance, "y*f(x)": margin, "position": position})
print(table.round(3).to_string(index=False))

fig, ax = plt.subplots(figsize=(6.5, 4.0))
ax.scatter(x1[y == 1],  x2[y == 1],  marker="o", label="y = 1")
ax.scatter(x1[y == -1], x2[y == -1], marker="s", label="y = -1")
x1_grid = np.linspace(-3.2, 3.2, 200)
ax.plot(x1_grid, (0 - 0.8*x1_grid)/0.6, "k-",  label="f(x) = 0")
ax.plot(x1_grid, (1 - 0.8*x1_grid)/0.6, "k--", label="f(x) = ±1")
ax.plot(x1_grid, (-1 - 0.8*x1_grid)/0.6, "k--")
inside = position == "inside"
ax.scatter(x1[inside], x2[inside], s=200, facecolors="none", edgecolors="red",label="inside margin")
for i in range(len(df)): ax.annotate(str(df["observation"].iloc[i]), (x1[i], x2[i]), fontsize=7)
ax.set_xlim(-3.2, 3.2); ax.set_ylim(-3, 3); ax.set_aspect("equal")
ax.set_xlabel("x1"); ax.set_ylabel("x2"); ax.legend()
fig.tight_layout(); fig.savefig(fig_dir / "q1-margin-plot.png", dpi=150)
plt.show()

# Counts per category:
counts=table["position"].value_counts()
# Misclassified observations are where the sign of f disagrees with y, i.e. y*f(x) < 0:
print(f"Counts per category:\n{counts.to_string()}")
misclassified = table.loc[margin < 0, "observation"].tolist()
print(f"Misclassified observations: {misclassified}")
```

(a) Distance to the boundary $|f(x)|/\parallel\beta\parallel$ (here $\parallel\beta\parallel=1$) and functional margin $yf(x)$ for each observation:

Obs	y	$f(x)$	Distance	$yf(x)$	Position
1	1	1.00	1.00	1.00	on
2	1	1.35	1.35	1.35	beyond
3	1	1.50	1.50	1.50	beyond
4	1	1.70	1.70	1.70	beyond
5	1	1.90	1.90	1.90	beyond
6	1	2.20	2.20	2.20	beyond
7	1	2.40	2.40	2.40	beyond
8	1	0.64	0.64	0.64	inside
9	1	0.20	0.20	0.20	inside
10	1	−0.20	0.20	−0.20	inside

Obs	y	$f(x)$	Distance	$yf(x)$	Position
11	−1	−1.00	1.00	1.00	on
12	−1	−1.30	1.30	1.30	beyond
13	−1	−1.55	1.55	1.55	beyond
14	−1	−1.75	1.75	1.75	beyond
15	−1	−1.95	1.95	1.95	beyond
16	−1	−2.15	2.15	2.15	beyond
17	−1	−2.45	2.45	2.45	beyond
18	−1	−0.64	0.64	0.64	inside
19	−1	−0.24	0.24	0.24	inside
20	−1	0.20	0.20	−0.20	inside

(b)

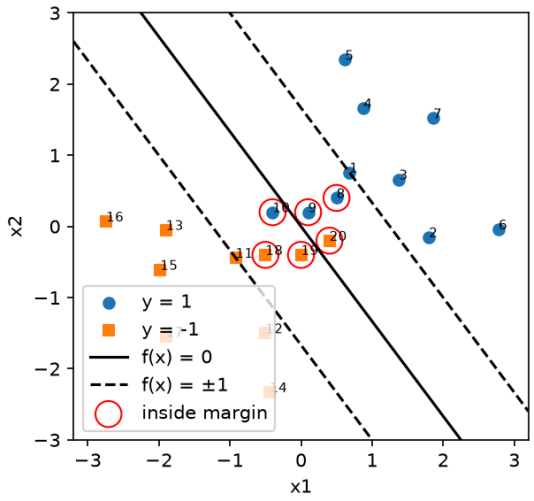


Figure 1: Observations, decision boundary $f(x) = 0$ (solid), and margin lines $f(x) = \pm 1$ (dashed); circled points are inside the margin

(c) 12 observations are beyond the margin, 2 are on the margin (observations 1 and 11), and 6 are inside the margin (observations 8, 9, 10, 18, 19, and 20). Observations 10 and 20 are misclassified, since $yf(x) = -0.20 < 0$: each lies on the wrong side of the decision boundary.

Question 2: Margin in four dimensions

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path
data_dir = Path(__file__).parent / "data"
fig_dir  = Path(__file__).parent / "figures"
df = pd.read_csv(data_dir / "margin-geometry-4d.csv")
X = df[["x1", "x2", "x3", "x4"]].to_numpy()
y = df["y"].to_numpy()

beta0 = -1
beta  = np.array([1, 2, -2, 4])
norm_beta = np.linalg.norm(beta)
u = beta / norm_beta
proj = X @ u
boundary = -beta0 / norm_beta
signed_dist = proj - boundary            # f / norm_beta
# score
f = beta0 + X @ beta

fig, ax = plt.subplots(figsize=(6.5, 2.5))
jitter = np.random.default_rng(0).uniform(-0.1, 0.1, len(y))
ax.scatter(proj[y == 1],  jitter[y == 1],  marker="o", label="y = 1")
ax.scatter(proj[y == -1], jitter[y == -1], marker="s", label="y = -1")
ax.axvline(boundary, color="k", linestyle="-", label="f(x) = 0")
M = np.min(np.abs(signed_dist))  # Geometric margin
ax.axvline(boundary + M, color="k", linestyle="--")
ax.axvline(boundary - M, color="k", linestyle="--")
ax.set_yticks([]); ax.set_xlabel("projection onto beta / ||beta||"); ax.legend()
fig.tight_layout(); fig.savefig(fig_dir / "q2-projection-plot.png", dpi=150)
plt.show()
closest = df["observation"][np.isclose(np.abs(signed_dist), M)].tolist()
print(f"Geometric margin M = {M:.3f}, closest observation = {closest}")

min_fm = np.min(y * f)
c = 1 / min_fm
beta0_svm = c * beta0
beta_svm  = c * beta
# Checks:
check1 =np.min(y * (beta0_svm + X @ beta_svm))   #should be 1
check2 = 1 / np.linalg.norm(beta_svm)             # should equal M from Step 3
print(f"Normalized coefficients: beta0 = {beta0_svm:.3f}, beta = {beta_svm}")
print(f"Check 1 (functional margin): {check1:.3f}")
print(f"Check 2 (geometric margin): {check2:.3f}")
```

(a) Each observation is projected onto the unit direction $\beta/\parallel\beta\parallel$, with $\parallel\beta\parallel = 5$. On this axis the decision boundary is at $-\beta_0/\parallel\beta\parallel = 0.2$.

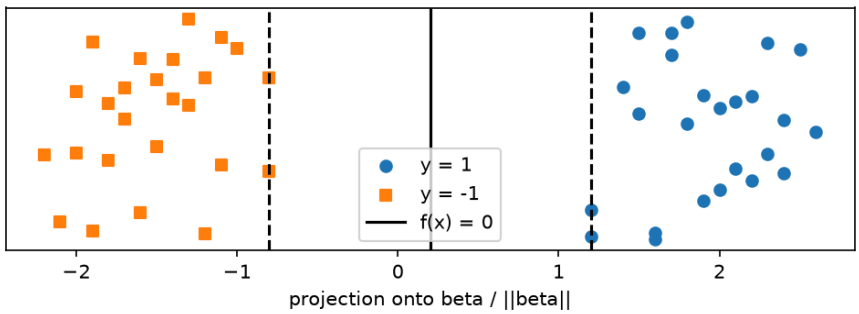


Figure 2: Projections onto $\beta/\parallel\beta\parallel$ (vertical jitter added for visibility), decision boundary (solid), and margin at distance M (dashed)

All 50 observations are correctly classified. The closest observations to the boundary are 4, 15, 33, and 36, so the geometric margin is $M = 1$.

(b) Dividing (β_0, β) by the smallest functional margin $\min_i y_i f(\mathbf{x}_i) = 5$ gives

$$\beta_0 = -0.2, \qquad \beta = (0.2, 0.4, -0.4, 0.8)^T.$$

The boundary is unchanged because both coefficients are multiplied by the same positive constant. With this normalization the smallest functional margin is 1, and $1/\parallel\beta\parallel = 1 = M$.

Question 3: Peer review of a Homework 7 skill

Using the skill

The assigned student's skill (`compare-lda-qda`, 2,929 bytes) was copied without changes from their Homework 7 report into a local `SKILL.md` file. The agent was given the path to this file and asked to follow it while analyzing the `SAheart` data, with response `chd` and all nine other variables as numerical predictors (`famhist` coded as Absent = 1 and Present = 2). The only other information provided was the random seed (4323), the language (Python), and an output folder. No preferences about methods, figures, model selection, or report style were given.

The run took about 72 seconds and used about 38,000 tokens. It finished on its own, so no steer message was needed. The skill reserved a stratified 30% test set (323 training, 139 test observations), compared LDA, QDA, and three probability blends with 5-fold cross-validation, and selected Blend $a = 0.75$.

Candidate	CV mistakes / 323	CV error
LDA	86	0.266
QDA	94	0.291
Blend $a = 0.25$	90	0.279
Blend $a = 0.50$	87	0.269
Blend $a = 0.75$ (selected)	**85**	**0.263**

The selected blend made 44 mistakes out of 139 on the test set (test error 0.317).

Peer review

GitHub Issue: https://github.com/LGONG0210/stat432-fall2026/issues/5

1- A single split makes the selection and test error unstable. One 5-fold split and a 139-case test set resulted in a test error of 25.9% (from the author's run) and 31.7% (from my run with seed 4323), leading to different chosen models. The author chose blend 0.5 (it was tied with 0.75, and the author chose the smaller blend weight), while mine chose blend 0.75.

2- The skill mentions changes like "optional PCA" and changing the predictors "unless a training-only check supports a change." However, it doesn't say what is this check and when to use it.

3- Add additional explanation to the results. The figure doesn't add anything the table doesn't already show. A plot of the blend weight vs. CV missclassification rate could offer more insight.